Logistic Regression with Missing Labels

AML project 1 - experimentation report

authors: Nadia Serafin, Barbara Seweryn, Zuzanna Sienko

Warsaw University of Technology

Source code available at: https://github.com/sienkozuzanna/logistic-regression-with-missing-labels

April 12, 2026

Contents

1 Datasets **2**

2 Missing labels schemas **2**

3 Discussion of correctness of implementation of FISTA algorithm **3**
3.1 Comparison with sklearn implementation 4

4 Methodology used in UnlabeledLogReg implementation **4**
4.1 kNN approach 4
4.2 EM (Expectation Maximization) approach 5

5 Analysis of experiments **5**
5.1 Experimental methodology 5
5.2 Experimantation results 6

A Appendix **9**
A.1 Dataset Characteristics 9
A.2 Spambase dataset 9
A.3 College dataset 9
A.4 Electricity dataset 10
A.5 Employee dataset 10

1 Datasets

For the testing process in the project 4 binary classification datasets were selected [1], [2]. These datasets are:

- Spambase - representing emails and whether or not they were assigned to spam,
- College scorecard - containing information about academic institutions and their completion rate as the target,
- Electricity - describing the energy prices dynamics,
- Employee turnover - containing information about employees at a technology company.

All datasets were subjected to exploratory data analysis, including inspection for missing values and encoding of the target variable into binary format. Highly correlated features were identified and removed to mitigate multicollinearity effects. The number of features reported in Table 4 corresponds to the datasets after preprocessing steps. Additionally, the College Scorecard dataset was used as a stratified subset to preserve class distribution.

2 Missing labels schemas

We considered four different missing data generation schemas applied to the target variable Y: MCAR, MAR1, MAR2, and MNAR. In all cases, the input consisted of fully observed pairs (X, Y), and the output was (X, Y_{obs}), where a fraction of labels in Y was replaced with a missing indicator (-1).

MCAR: A subset $\mathcal{M}$ of size $m = \lfloor n \cdot \text{missing_rate} \rfloor$ is sampled uniformly at random:

$$Y_{obs,i} = \begin{cases} -1 & i \in \mathcal{M} \\ Y_i & \text{otherwise} \end{cases}$$

MAR1: In this setting, the probability of missingness depends on a single explanatory variable (the first column of X). Let $X^{(1)}$ denote this variable. We define weights:

$$w_i = X_i^{(1)} - \min_j X_j^{(1)} + \varepsilon$$

and normalize them to obtain probabilities:

$$p_i = \frac{w_i}{\sum_{j=1}^{n} w_j}.$$

The subset $\mathcal{M}$ of size m is sampled without replacement according to the distribution p_i.

MAR2: Here, the missingness depends on all explanatory variables. First, the features are standardized:

$$\tilde{X}_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j + \varepsilon}.$$

The sampling probabilities are defined as

$$p_i = \frac{s_i - \min_j s_j + \varepsilon}{\sum_{k=1}^{n} (s_k - \min_j s_j + \varepsilon)},$$

where $s_i = \sum_{j=1}^{p} \tilde{X}_{ij}$ is the score of the i-th observation. The subset $\mathcal{M}$ is sampled according to p_i.

MNAR: In this setting, the probability of missingness depends on both the explanatory variables and the target variable. First, a feature-based score is computed as in MAR2 and denoted by $s_i^{(X)}$.

Additionally, a label-dependent component is introduced:

$$s_i^{(Y)} = \begin{cases} \gamma_1 & \text{if } Y_i = 1 \\ \gamma_0 & \text{if } Y_i = 0, \end{cases}$$

where $\gamma_1 = 2.0$ and $\gamma_0 = -0.5$ are fixed hyperparameters controlling the class-dependent shift in missingness. A positive value of γ_1 increases, while a negative value of γ_0 decreases, the probability of missing labels. The final score is defined as:

$$s_i = s_i^{(X)} + s_i^{(Y)}.$$

The sampling probabilities are then given by

$$p_i = \frac{s_i - \min_j s_j + \varepsilon}{\sum_{k=1}^{n} (s_k - \min_j s_j + \varepsilon)},$$

and the subset $\mathcal{M}$ of size m is sampled without replacement according to p_i.

3 Discussion of correctness of implementation of FISTA algorithm

In the implementation for logistic Lasso regression, the FISTA algorithm was used. It is a method for minimizing a function which is a sum of two convex functions f and g such that $F(x) = f(x) + g(x)$, where f is a smooth function and g is a nonsmooth function - non-differentiable. This is general assumption for the model analyzed in the article [3].

In this case, the function f corresponds to logistic loss, while g is defined as the L_1 penalty applied to the coeffictient vector.

The implementation follows the quadratic approximation of $F(x) = f(x) + g(x)$ at a given point y:

$$Q_L(x, y) = f(x) + < x - y, \nabla f(y) > + \frac{L}{2}||x - y||^2 + g(x).$$

The next iterate is obtained by minimizing this function, leading to the operator:

$$p_L(y) = \arg\min_x Q_L(x, y).$$

As shown in the paper, for L_1 regularization this operation reduces to a soft-thresholding (shrinkage) operator applied element-wise to the coefficients. This step is implemented in the function `_gradient_step_L1_part` responsible for handling the non-differentiable part of the objective.

The gradient for the smooth component f is computed in a standard way in the function `_compute_gradient`:

$$\nabla f(\beta) = X'(Y - p), \quad p = \sigma(X\beta').$$

This separation is consistent with the structure of FISTA, where the non-smooth part is not included in the gradient but instead addresed in the operator for minimization.

To handle the non-differentiable L_1 penalty, soft-thresholding operator is introduced:

$$T_\alpha(x)_i = (|x_i| - \alpha)_+ \operatorname{sign}(x_i),$$

which is applied element-wise. This is implemented in `_gradient_step_L1_part` function.

A key part of FISTA algorithm is how he algorithm updates the iterate points y_k:

$$x_k = p_L(y_k), \quad t_{k+1} = \frac{1 + \sqrt{1 + 4t_k^2}}{2}, \quad y_{k+1} = x_k + \frac{t_k - 1}{t_{k+1}}(x_k - x_{k-1})$$

This formulation is implemented in the `fit` function.

In the aritcle the constant step size is chosen based on the Lipschitz constant of ∇f - the gradient of the smooth part f of the objective function. In our case, we consider the Lipschitz constant of the gradient of the smooth part of the objective function which is logistic regression loss.

To determine the default step size in FISTA, we need an upper bound on the Lipschitz constant of the gradient. For this purpose, we analyze the derivative of the gradient. For this purpose, we analyze the derivative of the gradient, i.e. the Hessian matrix $\nabla^2 f(\beta) = -X' \frac{\partial p}{\partial \beta}$ and since $p = \sigma(X\beta')$, the derivative of the sigmoid equals $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Thus for the vector p we obtain $\frac{\partial p}{\partial \beta} = WX$, where W is a diagonal matrix of the form $W = diag(p_i(1 - p_i))$. And substituting this into the Hessian gives $\nabla^2 f(\beta) = -X'WX$. If the loss is written with averaging over observations, then the Hessian takes the form $\nabla^2 f(\beta) = \frac{1}{n}X'WX$.

For the Lipschitz constant, only the magnitude matters, so we consider the bound

$$||\nabla^2 f(\beta)|| \leq \frac{1}{n}||X'WX||.$$

For $p_i \in (0, 1)$: $p_i(1 - p_i) \leq \frac{1}{4}$.
The maximum is attained for $p_i = \frac{1}{2}$, therefore: $W \leq \frac{1}{4}I$.
Using this bound, we obtain: $\nabla^2 f(\beta) \leq \frac{1}{4n}X'X$
and the gradient of the logistic loss is Lipschitz continuous with constant bounded by $L \leq \frac{\lambda_{max}(X'X)}{4n}$
and we get

$$\lambda_{max}(X'X) = ||X||_2^2 = L \leq \frac{||X||_2^2}{4n},$$

which gives the default step size in the FISTA implementation:

$$\eta = \frac{1}{L}$$

included in `_compute_default_step_size` function.

In addition, the implementation includes automatic selection of the optimal regularization parameter λ based on a chosen evaluation metric computed on the test set. For each λ, the model is trained and evaluated, and the best value is selected. The final model performance is then evaluated using multiple metrics (recall, precision, F1-score, balanced accuracy, ROC AUC, and average precision) and the results are visualized using plots of metric values and coefficient paths as functions of λ. Detailed plots for all datasets are provided in Appendix A.2 and results for each metric - in comparison Table 1.

Each iteration of the algorithm has computational complexity $O(np)$ due to matrix-vector multiplications and FISTA achieves an accelerated convergence rate of $O(1/k^2)$.

3.1 Comparison with sklearn implementation

The custom FISTA implementation was compared with the `sklearn` logistic regression model. The same training procedure, type of penalty for regularization and regularization grid were used to ensure a fair comparison. The selection of the optimal regularization parameter λ was performed based on a chosen evaluation metric - ROC AUC.

Across all datasets, both approaches achieved very similar performance. The differences in metrics were marginal. In most cases, the `sklearn` implementation obtained slightly higher scores, but the gap remained small. However, sometimes custom implementation with FISTA was better.

The number of non-zero coefficients was also comparable, confirming that both methods produce similar sparsity patterns. Differences in the selected regularization parameter and coefficient values were observed, which is expected due to differences in optimization procedures.

Overall, the results confirm that the implemented FISTA algorithm is correct and produces results consistent with a standard library implementation.

Table 1: Comparison of the custom FISTA implementation and the `sklearn` implementation across datasets. The optimal regularization parameter was selected using ROC AUC.

Dataset	Method	Recall	Precision	F1-score	Balanced Accuracy	ROC AUC	Avg Precision	Best λ	Non-zero Coef.
Spam	FISTA	0.885	0.902	0.893	0.910	0.964	0.916	0.000100	50
Spam	sklearn	0.904	0.894	0.899	0.915	0.967	0.929	0.023357	55
College	FISTA	0.666	0.788	0.722	0.820	0.963	0.828	0.000100	87
College	sklearn	0.680	0.793	0.732	0.827	0.963	0.830	0.042813	117
Electricity	FISTA	0.606	0.776	0.680	0.739	0.815	0.785	0.000100	7
Electricity	sklearn	0.608	0.773	0.681	0.739	0.815	0.785	0.003793	8
Employee	FISTA	1.000	0.986	0.993	0.500	0.687	0.993	0.000183	6
Employee	sklearn	1.000	0.986	0.993	0.500	0.687	0.993	5.455595	6

4 Methodology used in UnlabeledLogReg implementation

We considered three approaches for learning with missing labels: the proposed *UnlabeledLogReg*, the *Naive* baseline, and the *Oracle* reference model. Logistic regression was implemented using an L1-regularized formulation optimized by FISTA.

UnlabeledLogReg. The proposed method was trained on partially observed data (X, Y_{obs}). Two strategies for label completion were considered: (i) kNN-based imputation and (ii) an EM-based iterative procedure, both producing a completed label vector used for training logistic regression.

Naive baseline. The Naive approach trained logistic regression only on observations with known labels, i.e., samples for which $Y_{obs} \neq -1$, discarding all unlabeled data.

Oracle model. The Oracle model was trained on fully observed data (X, Y) and served as a baseline.

4.1 kNN approach

The first, and more trivial, approach to address the instances with missing label that was developed was focused on assigning the labels to the unlabeled instances using the knowledge from the labeled instances. One of the natural strategies in this area was to compare the instances with missing labels to other similar instances that do have a label assigned.

To follow that intuition we utilized the k-Nearest Neighbors (kNN) model [4]. When the UnlabeledLogReg model is fit to the train dataset, using this approach, the dataset was split into two components: one with instances with assigned

labels, and another one with the unlabeled instances. Following than a kNN model, with distance weighting and a specified number of neighbors to be considered, was fit to the component with labeled instances. Subsequently the kNN model was then used to assign labels to the instances in the second component.

Two instances of this method were implemented. The first one, later referenced to as 'hard kNN', takes the specified number of neighbors into account and performs label voting, weighted by the distance from the neighbor to the instance. The produces label is when either 0 or 1, depending on weather the weighted average of votes, surpasses the 0.5 threshold. The second method, performs the same weighted voting however, it assigns the value of the weighted average of votes as the final label, therefore incorporating the certainty of the label assignment. The pseudocode of the procedure is summarized in Algorithm 1.

Algorithm 1 kNN procedure for UnlabeledLogReg

Input: (X, Y_{obs}) where $Y_{obs} \in \{0, 1, -1\}, method = \{'hard', 'proba'\}$
Split data into labeled and unlabeled subsets: $X_L, Y_L = \{(X_i, Y_i) : Y_{obs,i} \neq -1\}$, $X_U = \{X_i : Y_{obs,i} = -1\}$
Train kNN model on (X_L, Y_L) with the chosen method
Predict labels Y_U for X_U using kNN model
Construct full label vector:
$Y_{full} \leftarrow Y_{obs}$
Replace missing entries with Y_U
Train logistic regression (FISTA) on (X, Y_{full})
Output: trained logistic regression model

4.2 EM (Expectation Maximization) approach

The idea of the EM algorithm [5] is to address learning problems with incomplete data by iteratively estimating the missing values and optimizing the model parameters.

The algorithm consists of two main steps: the E-step (Expectation) and the M-step (Maximization). In our approach, this corresponds to iteratively predicting missing labels using the current model (E-step) and updating the model based on these estimates (M-step). The pseudocode of the procedure is summarized in Algorithm 2.

Algorithm 2 EM procedure for UnlabeledLogReg

Input: (X, Y_{obs}) where $Y_{obs} \in \{0, 1, -1\}$
Split data into labeled and unlabeled subsets: $X_L, Y_L = \{(X_i, Y_i) : Y_{obs,i} \neq -1\}$, $X_U = \{X_i : Y_{obs,i} = -1\}$
Train initial logistic regression model on (X_L, Y_L)
while not converged and iteration limit not reached **do**
    E-step:
    Predict probabilities for unlabeled samples:
    $Y_U \leftarrow$ model.predict_proba(X_U)
    Construct full label vector:
    $Y_{full} \leftarrow Y_{obs}$
    Replace missing entries with Y_U
    M-step:
    Retrain logistic regression (FISTA) on (X, Y_{full})
    Update model parameters
    Check convergence based on change in parameters
end while
Output: trained logistic regression model

5 Analysis of experiments

5.1 Experimental methodology

The experiments were conducted using stratified cross-validation with 5 folds. In each split, the data was divided into training, validation, and test sets. Missing labels were generated only in the training part, while validation and test sets

remained fully observed. Four missing data schemas were considered: MCAR, MAR1, MAR2, and MNAR, each applied with different missing rates $c \in \{0.1, 0.3, 0.5, 0.8\}$.

For each combination of dataset, missing mechanism, and missing rate, all methods were trained and evaluated across cross-validation folds, which resulted in a large number of experiments ($1600 = 4$ datasets $\times$ 4 missing schemas $\times$ 4 missing rates $\times$ 5 folds $\times$ 5 methods). Due to the computational cost, especially for high-dimensional datasets, the experiments were executed in parallel to reduce runtime. The following evaluation metrics were collected: accuracy, balanced accuracy, F1-score, and ROC AUC, which were then averaged across folds.

5.2 Experimantation results

The performance achieved by the Oracle method, which we use as the baseline for all other experiments is presented in Table 2. The Oracle method represents a model trained on the entire datasets, without the missing label instances.

Table 2: Model performance metrics across datasets (Oracle method)

Dataset	Accuracy	Balanced Accuracy	F1-score	ROC AUC
Spam	0.920	0.912	0.896	0.969
College	0.930	0.813	0.706	0.957
Electricity	0.756	0.735	0.676	0.819
Employee	0.986	0.500	0.993	0.686

As it can be seen in Table 2, the accuracy and balanced accuracy metric values for the Employee Turnover dataset are respectively equal to 0.986 and 0.5. The Employee Turnover dataset is extremely unbalanced, with 33958 instances of class 1 and 494 instances of class 0. For all other experiments conducted for this dataset, with the missing labels, the balanced accuracy value is also equal to 0.5, while the standard accuracy metric is equal to almost 1. This shows that not only the Oracle method, but all experiments on this dataset, produced a model that was unable to detect the 0-class instances, and was only predicting the majority class. Therefore all examined approaches did not handle this extremely unbalanced problem. Due to this issue, the Employee dataset was excluded from further analyses, as it does not contribute any information.

The further analyses, performed on the Spam, Electricity and College datasets, were focused on comparing the performance of the three missing label modeling methods under four missing data schemas and in respect to the missing rate value. The models that achieved the best performance (in terms of balanced accuracy) for each dataset, missing rate and missing schema are presented in Table 3.

Table 3: Models with the best balanced accuracy across datasets, missing schemas and rates. The values of the metric for each model are written in parentheses next to the model name. The values in parentheses next to dataset names are the baseline values (from Oracle method).

Dataset	Missing rate	MCAR	MAR1	MAR2	MNAR
Spam (0.912)	0.1	Naive (0.918)	Naive (0.910)	Naive (0.916)	Naive (0.912)
	0.3	Naive (0.915)	Naive (0.904)	Naive (0.917)	Naive (0.914)
	0.5	Naive (0.914)	Naive (0.899)	Naive (0.911)	Naive (0.908)
	0.8	Naive (0.903)	Naive (0.894)	Naive (0.898)	Naive (0.891)
College (0.813)	0.1	Naive (0.811)	KNN_proba (0.810)	KNN_hard (0.812)	Naive (0.810)
	0.3	Naive (0.808)	KNN_hard (0.808)	EM (0.812)	Naive (0.813)
	0.5	Naive (0.803)	Naive (0.797)	Naive (0.798)	KNN_hard (0.788)
	0.8	Naive (0.790)	Naive (0.685)	Naive (0.753)	Naive (0.755)
Electricity (0.735)	0.1	EM (0.735)	EM (0.735)	Naive (0.735)	KNN_hard (0.734)
	0.3	Naive (0.735)	Naive (0.734)	Naive (0.735)	EM (0.730)
	0.5	EM (0.735)	Naive (0.733)	Naive (0.734)	Naive (0.726)
	0.8	Naive (0.735)	Naive (0.712)	Naive (0.733)	Naive (0.717)

The Naive approach demonstrates high stability on the Spam dataset, consistently outperforming other methods across all settings. This suggests that the Spam dataset is less sensitive to missing labels. In contrast, for the College and Electricity datasets, the best-performing method varies depending on the missing data mechanism and rate, indicating

higher sensitivity to data characteristics. An interesting observation to be made is that there are cases shown in Table 3 where the models achieved a better balanced accuracy score than the baseline model. As our experimentation has been performed on 5-fold stratified cross-validation, it may be simply the case of random variation. However since the models were trained on identical folds, it may suggest that the missing label handling process introduced regularization into the models and therefore caused better generalization in some of the them.

Among the missing data schemas, MAR1 appears to be the most challenging, as it leads to the largest average performance degradation, particularly at higher missing rates. In contrast, MCAR consistently results in the smallest performance loss. The degradation of ROC AUC compared to the baseline, for the MAR1 schema, is presented in Figure 1. There is a significant difference in performance degradation for the missing rate equal to 0.8 compared to other tested values. The observed differences in performance degradation between methods across datasets suggest that the effectiveness of missing label handling techniques is highly dependent on the underlying data. In respect to ROC AUC degradation the Spam dataset produced the most consistent results for all approaches. However a visible difference between kNN methods and EM method can be noted for the Electricity and College datasets. For the College dataset, the EM method shows the largest performance degradation for higher missing rate (0.8), which may indicate that its underlying probabilistic assumptions do not align well with it. The kNN-based methods show relatively high degradation for most cases, especially for the Electricity dataset. This may be a consequence of their reliance on local neighborhood structure, which becomes unreliable as the amount of missing data increases. The sharp increase in degradation at the missing rate of 0.8 suggests a threshold effect, where the amount of available label information becomes insufficient for building a reliable model, regardless of the method used.

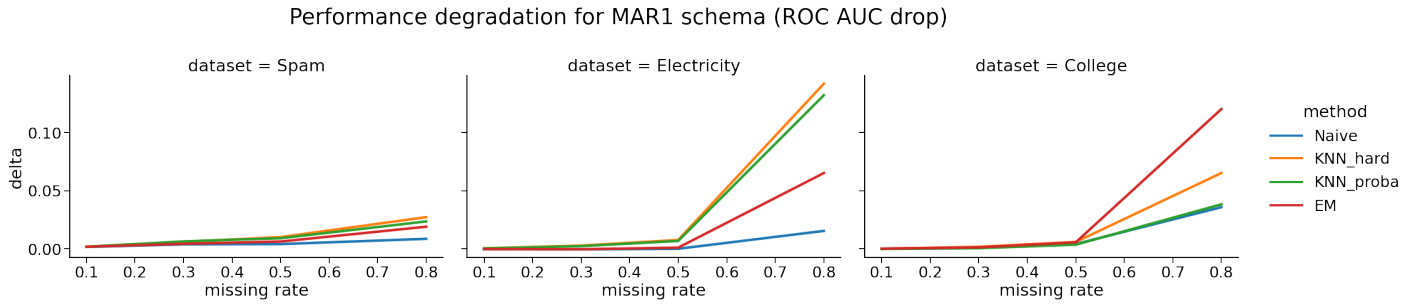


Figure 1: Performance degradation of different methods for MAR1 schema. Delta is calculated as the difference between the ROC AUC of the baseline model and the ROC AUC for the particular model.

In Figure 2 it can be noticed that the distribution of balanced accuracy for 0.8 missing rate differs significantly over the missing schemas, with the biggest performance drop observed for MAR1 and MNAR. It also suggests that the most stable overall performance is achieved by the Naive approach.

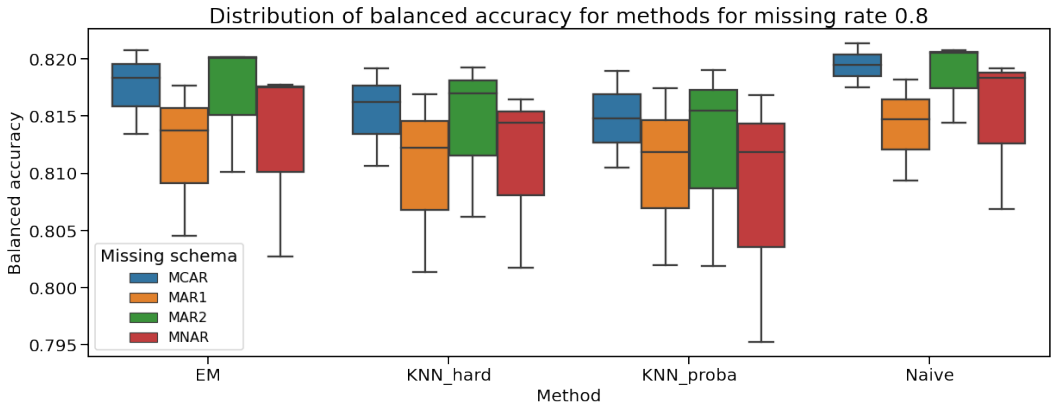


Figure 2: Distribution of the balanced accuracy metric for each method for missing rate equal to 0.8

The results indicate that no single method is uniformly robust across datasets, and their performance depends on whether the data structure is better captured by probabilistic models (EM) or local similarity approaches (kNN).

References

[1] D. Dua and C. Graff, *UCI machine learning repository*, 2019.

[2] J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked science in machine learning," *SIGKDD Explorations*, vol. 15, pp. 49–60, 2013.

[3] A. Beck and M. Teboulle, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM journal on imaging sciences*, vol. 2, no. 1, pp. 183–202, 2009.

[4] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theor.*, vol. 13, no. 1, pp. 21–27, Sep. 2006, ISSN: 0018-9448.

[5] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *Journal of the Royal Statistical Society: Series B*, vol. 39, pp. 1–38, 1977.

List of Figures

1 Performance degradation of different methods for MAR1 schema. Delta is calculated as the difference between the ROC AUC of the baseline model and the ROC AUC for the particular model. 7
2 Distribution of the balanced accuracy metric for each method for missing rate equal to 0.8 7
3 Detailed optimization plots for the Spambase dataset. 9
4 Detailed optimization plots for the College dataset. 9
5 Detailed optimization plots for the Electricity dataset. 10
6 Detailed optimization plots for the Employee dataset. 10

List of Tables

1 Comparison of the custom FISTA implementation and the `sklearn` implementation across datasets. The optimal regularization parameter was selected using ROC AUC. 4
2 Model performance metrics across datasets (Oracle method) 6
3 Models with the best balanced accuracy across datasets, missing schemas and rates. The values of the metric for each model are written in parentheses next to the model name. The values in parentheses next to dataset names are the baseline values (from Oracle method). 6
4 Dataset statistics after exploratory data analysis. 9

A Appendix

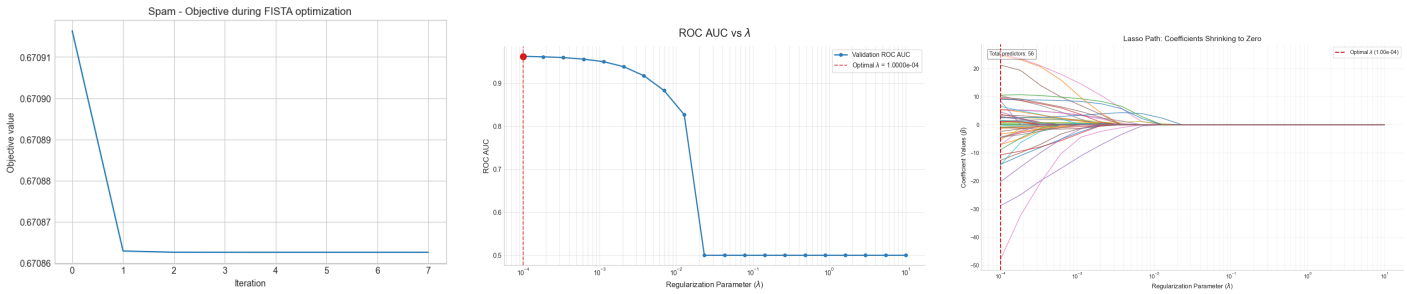
This appendix contains dataset characteristics and detailed optimization plots for all datasets, including objective function values, validation ROC AUC as a function of λ and coefficient paths during FISTA training.

A.1 Dataset Characteristics

Table 4: Dataset statistics after exploratory data analysis.

Dataset	Instances	Features	Dropped features	Missing values	Source
Spambase	4601	56	1	None	UCI (id = 94)
College Scorecard (Subset)	12469	114	3	None	OpenML (id = 46805)
Electricity	38474	7	0	None	OpenML (id = 44120)
Employee Turnover	34452	9	0	None	OpenML (id = 43551)

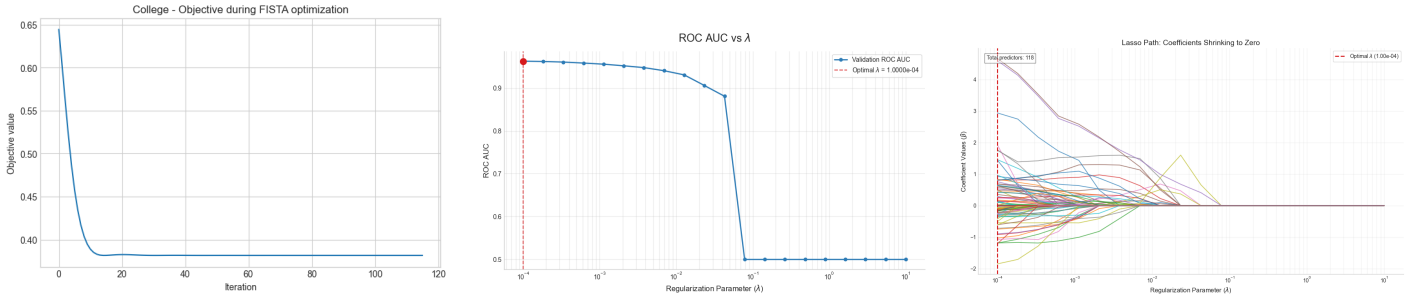
A.2 Spambase dataset



(a) Objective value (b) ROC AUC vs λ (c) Coefficient paths

Figure 3: Detailed optimization plots for the Spambase dataset.

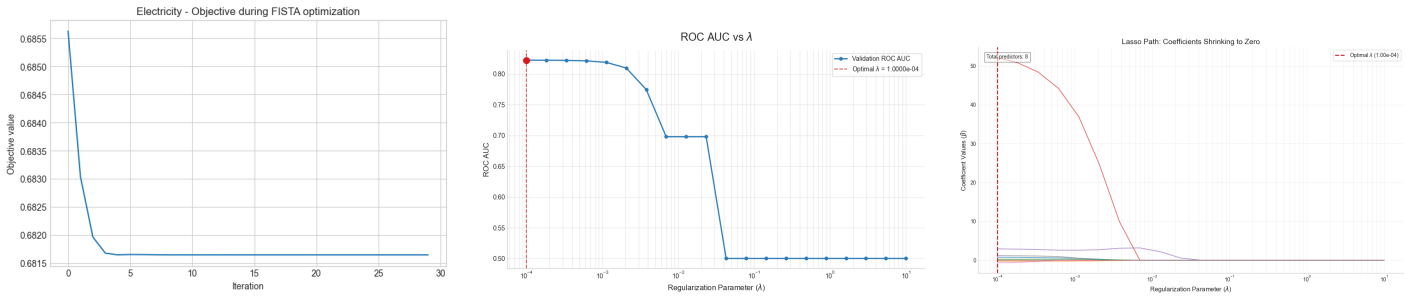
A.3 College dataset



(a) Objective value (b) ROC AUC vs λ (c) Coefficient paths

Figure 4: Detailed optimization plots for the College dataset.

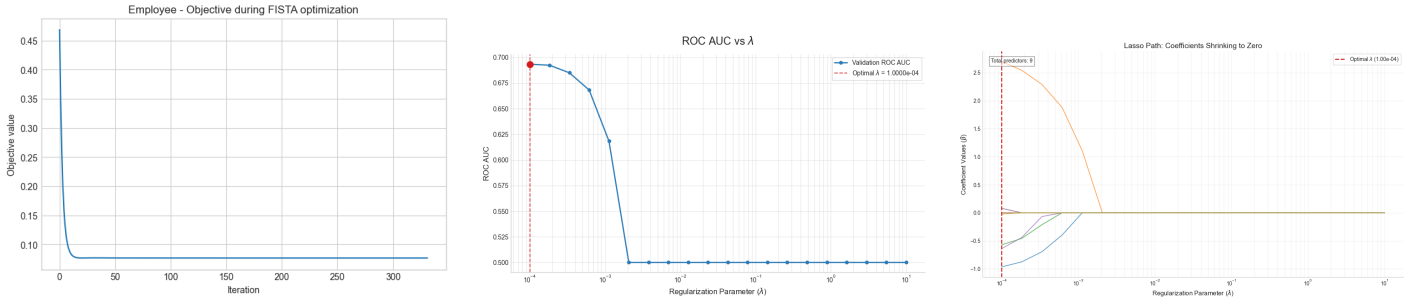
A.4 Electricity dataset



(a) Objective value (b) ROC AUC vs λ (c) Coefficient paths

Figure 5: Detailed optimization plots for the Electricity dataset.

A.5 Employee dataset



(a) Objective value (b) ROC AUC vs λ (c) Coefficient paths

Figure 6: Detailed optimization plots for the Employee dataset.